Safeguarding User Data Privacy in Online Large Language Model Services

Tianyu Bai, Yunhe Feng, Song Fu

[a]*Department of Computer Science and Engineering, University of North Texas, , Denton, 76203, TX, USA*

Abstract

Large Language Models (LLMs), such as GPT, have become central to modern AI applications, including conversational agents, language translation, and document processing. Due to their computational demands, these models are typically hosted on remote servers, requiring users to transmit potentially sensitive data for inference. This raises serious privacy concerns, as user inputs may contain personally identifiable information (PII) and are vulnerable to misuse or unauthorized retention.

To address this challenge, we present PPGPT, a novel and practical privacy-preserving GPT framework. PPGPT employs additive secret sharing to protect user input by enabling secure inference on secret shares rather than raw data. We design secure versions of key transformer components, including GELU and Softmax layers using Beaver's triples and Taylor series, and introduce an optimized secure layer normalization protocol to reduce overhead.

Experimental results show that PPGPT achieves comparable generation quality to the base model, with a negligible logits difference of 10^{-5} and an average inference time of 1.98 seconds. The framework is lightweight, generalizable to transformer-based LLMs, and suitable for deployment in real-world online services requiring strong privacy guarantees.

Keywords: Large Language Models, Data Privacy, Online Services, Enhanced Transformers

1. Introduction

Large Language Models (LLMs), such as GPT [2], are revolutionizing fields like healthcare and finance with their natural language capabilities.

August 23, 2025

Widely adopted in both commercial (e.g., ChatGPT, Gemini, Watson) and open-source (e.g., LLaMA) platforms, these models are typically hosted on remote servers due to their high computational demands. However, this setup requires users to transmit sensitive data to third-party servers, raising critical privacy concerns. The server-side processing of user queries introduces risks of data misuse, leakage, or unauthorized retention, especially in privacy-sensitive applications.

Privacy risks are prevalent during data processing, particularly when the trustworthiness of provider servers cannot be fully guaranteed. Internal breaches and cyberattacks targeting back-end systems pose serious threats to user data confidentiality. Deep learning models operate directly on user data, maintaining access throughout execution. Furthermore, LLM service providers may retain user data for future training to improve model accuracy, leading to potential data exposure during storage, retraining, or even as unintended output to other users.

Existing encryption methods such as fully homomorphic encryption [3, 4] offer theoretical security during computation but suffer from significant latency. Differential privacy [5, 6, 7, 8, 9, 10] introduces noise to protect sensitive information but can reduce model accuracy. Similarly, masking techniques like InfoMask [12, 13, 14, 15] effectively safeguard training data but offer limited protection during inference.

To address these challenges, we propose a novel framework that enhances LLMs like GPT to protect user privacy by developing a secret sharing protocol. Our work introduces a novel approach by leveraging a combination of cryptographic techniques, including additive secret sharing, Beaver's tuples, and Taylor series, to design privacy-preserving Transformer layers. The framework not only ensures privacy protection but also achieves lightweight implementation and faster execution times through targeted optimizations on Transformer layers. These Transformer layers, commonly composed of linear transformations, activation functions, and normalization steps, are adapted in our framework to operate securely under the constraints of secret sharing without compromising model accuracy. As a result, our method enables secure inference directly on ciphertext, delivering accurate outputs with relatively low latency while maintaining ease of implementation. This balance of privacy, efficiency, and practicality underscores the distinctiveness of our approach.

In our approach, user data is divided into ciphertext, with each portion processed independently by AI models with secure functions on different

servers. This enhances data privacy, as no raw data can be retrieved from any single party. The results are then aggregated without exposing raw data throughout the entire process.

We introduce Privacy Preserving GPT (PPGPT), a lightweight implementation of GPT, to demonstrate the efficacy of our solution as a proof of concept. By incorporating the techniques listed in the paragraph into the original GPT, we achieve a balance between privacy preservation and computational efficiency. Our research delves into the design, implementation, and performance evaluation of PPGPT and its critical layers, offering insights into their potential to advance privacy-preserving AI technologies in LLM applications and systems.

The main contributions of this paper are as follows:

- We introduce a methodology for enhancing LLM models to protect the privacy of user data. Our proposed method is generic and can be applied to any transformer-based LLMs to enable privacy preservation.

- We detail the development of secure functions for LLMs, emphasizing privacy preservation on LLM servers. A crucial aspect of PPGPT is the integration of multiple cryptographic technologies within the design of its secure layers. This paper delves into the details of these secure functions, elucidating their pivotal role in maintaining privacy during data processing on the server side.

- We have conducted comprehensive experiments to assess the effectiveness and performance of PPGPT. On the user side, we have measured the time required to generate tokens from user input data and create secret shares. On the server side, we have evaluated the time needed to generate outputs from the secret shares and compared the execution times of individual layers in the original LLM. Specifically, we compared the layers with secure functions against their original non-secure versions. PPGPT has an average difference of e^{-5} in the inference output compared to the base model, which is negligible in terms of its impact on text generation results. The secure functions in the LLM model result in a prolonged inference time of 1.98s and generate an additional 74 bytes of data compared to the base model.

2. Background and Threat Model

2.1. Transformers in LLMs

Transformers [16, 17, 18, 26, 20, 21, 22] have emerged as a breakthrough in LLMs, revolutionizing the field with its attention mechanism and parallelizable architecture. Figure 1 demonstrates the general execution process of GPT text generation. User input text data is tokenized using an encoder to transform it into numerical representations. These numerical tokens are then mapped to dense vectors with spatial information via token embedding and position embedding.

Next, the vector data is passed to the deep neural network (DNN) model, which consists of two major components: sequential self-attention Transformer blocks and feed-forward networks (FFNs) [23, 24, 25]. Self-attention (right side in Figure 1) [26, 27, 28] in the Transformer model allows it to weigh the importance of each token in a sequence relative to others by computing attention scores through query, key, and value vectors for each token, and using these scores to produce a weighted sum of the value vectors. This mechanism captures dependencies and contextual information from the entire sequence, enhancing the model's understanding of language patterns. FFNs capture complex features and apply non-linear transformations and projections to the token representations. Together, these components enable the model to produce meaningful and human-readable text based on the input. Finally, the next generated token is selected based on the model's probability output, and the generated tokens are decoded into output words.

2.2. Threat model

In this study, we adopt the Honest But Curious model [29]. This model establishes a trust relationship where the data owner is considered trustworthy, while the PPGPT server is deemed untrusted. The GPT text generation process, detailed in the previous section, runs entirely on the server side and takes raw user's text data as input. Additionally, there is no guarantee that user's input data is not copied and stored on the server side for analysis and training purposes.

Our primary focus is on safeguarding the privacy of user input text data and the corresponding output response data generated by the GPT model. We introduce a privacy-preserving LLM based on the philosophy of secret sharing, wherein the raw data is split and combined only on the user side, ensuring no privacy is compromised during the server execution process.

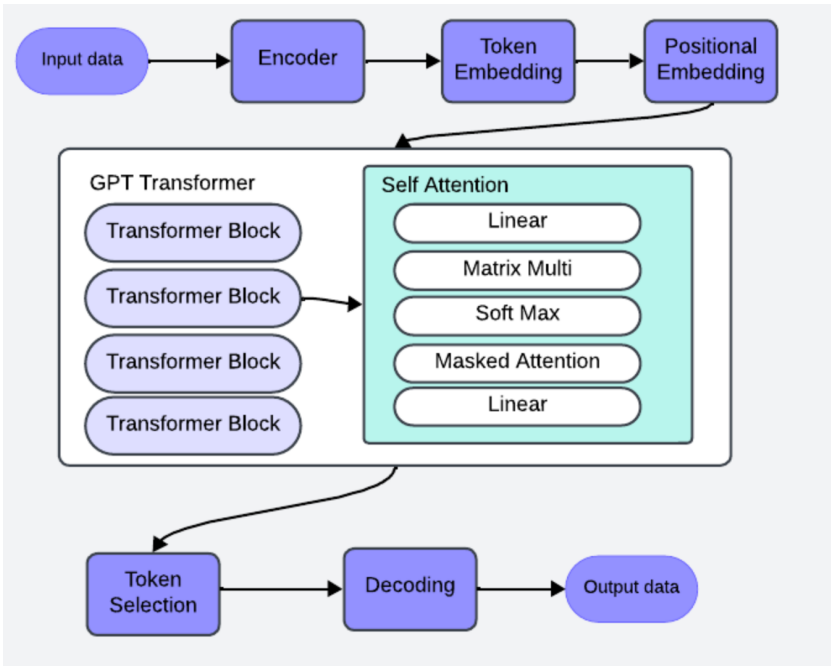

Figure 1: GPT Architecture and Execution Flow

2.3. Additive Secret Sharing

The theoretical framework of additive secret sharing provides the basis for designing secure functions tailored to process encrypted data shares within PPGPT infrastructure.

Originally proposed to enhance security measures [30], secret sharing operates as follows: a trusted entity divides a secret into N shares, with each participant holding one share. Reconstruction of the original secret requires the collaboration of at least m participants (where $m \leq N$). The primary operations involved in secret sharing are the *divide* and *assemble* functions, as depicted in Equation 1.

$$\begin{aligned} &DEV(\boldsymbol{k}) = (\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots, \boldsymbol{k}_n), \quad \boldsymbol{k} \in V \\ &ASS(\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots, \boldsymbol{k}_n) = \boldsymbol{k} \end{aligned} \tag{1}$$

The theory of additive secret sharing [31, 32, 33, 34] introduces two constraints: 1) secret shares maintain an additive relation,

$$\boldsymbol{k} = (\boldsymbol{k}_1 + \boldsymbol{k}_2 + \cdots + \boldsymbol{k}_n) \mod p$$

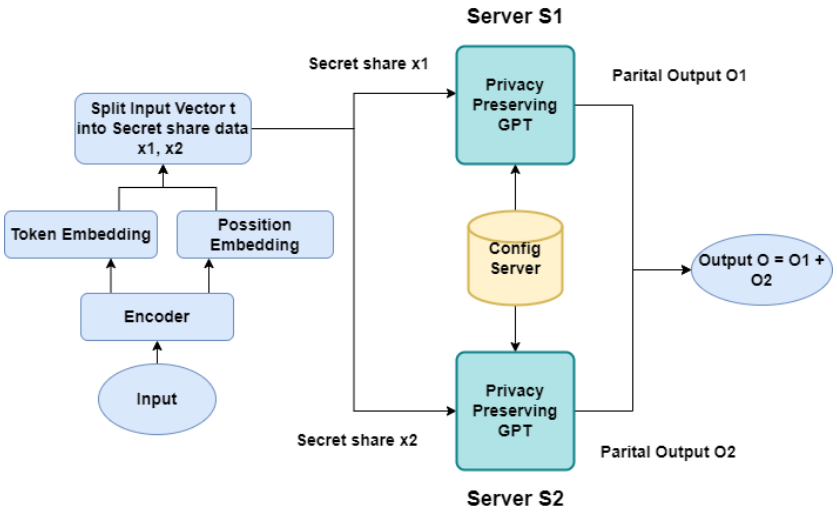

Figure 2: PPGPT Architecture

2) the number of shares (m) equals the participants (N), so a secret k can only be reconstructed by combining all shares. This enables secure operations on ciphertext.

In the context of Large Language Models (LLMs) with two parties, the embedded tokens $\boldsymbol{t}$ can be securely divided into two secret shares. First, a random vector $\boldsymbol{x}^{(1)}$ is generated with the same shape as $\boldsymbol{t}$, serving as the first secret share. The second secret share $\boldsymbol{x^{(2)}}$ is then computed as $\boldsymbol{x^{(2)}} = (\boldsymbol{t} - \boldsymbol{x^{(1)}}) \mod p$. This process ensures that the original token vector T can only be reconstructed by combining both secret shares, using $\boldsymbol{t} = (\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}}) \mod p$.

The randomness of the secret shares and their additive relationship enable the design of a system capable of performing LLM computations on ciphertexts, without revealing the value of the original input text data.

The randomness of the secret shares and their additive relationship enable the design of a system capable of performing LLM computations on ciphertexts, without revealing the value of the original input text data.

3. Architecture and Design

3.1. PPGPT Architecture

Figure 2 illustrates the architecture of the privacy-preserving GPT. The primary difference from the traditional GPT architecture is twofold: first,

it involves separating tasks between the user and server sides; second, the original GPT's single LLM server that processes raw user data inputs is replaced by multiple servers that process secret shares as input. This design aims to safeguard the privacy of both input and output text data throughout every stage of processing.

The key components of a privacy-preserving GPT system consist of the data owner, who provides input user data; the pre-processing model, which consists of an encoder tasked with generating tokens from the input, and embedding models that create vector representations of these tokens and split tokens into secret shares; PPGPT servers, which host the privacy-preserving transformers; and Config server, which supplies secure protocol parameters and facilitates key generation.

The reason for adopting a pre-processing model on the client side rather than on the server side is driven by the following concerns: First, to avoid sending raw data to the server, our secure protocol involves splitting user data into random secret shares by first generating a random vector $\boldsymbol{x}^{(1)}$, then the second secret share $\boldsymbol{x}^{(2)}$ is computed by subtracting the random vector from the original token $\boldsymbol{t}$, as $\boldsymbol{x}^{(2)} = (\boldsymbol{t} - \boldsymbol{x}^{(1)})$.

However, since the tokenization and embedding processes are mapping operations, random values are not guaranteed to map to the exact same tokens. To address this, tokenization is performed before the data input is split into secret shares. Second, client-side computational resources are often more limited than server-side resources. The pre-processing model is kept lightweight because it does not involve deep learning computations. Both the encoder and embedding models are available in open-source implementations, and similar models can be used across different platforms. Additionally, while the user's input data is numerized after tokenization, decoding the token vectors into the original text is straightforward if the encoding and embedding methodologies are known. Therefore, secure functions in privacy-preserving transformers still require secret shares of the generated tokens.

On the server side, for each PPGPT model (see Figure 3), the architecture mirrors that of the original GPT model but with each deep neural network (DNN) layer enhanced to handle ciphertext inputs securely. The model retains the same number and types of layers as the original GPT model.

The model includes a variable number of secure transformer blocks, typically determined by the size of the implementation. Non-linear layers may require secure parameter exchanges with other PPGPT servers, as secure functions in the privacy-preserving transformer need intermediate computa-

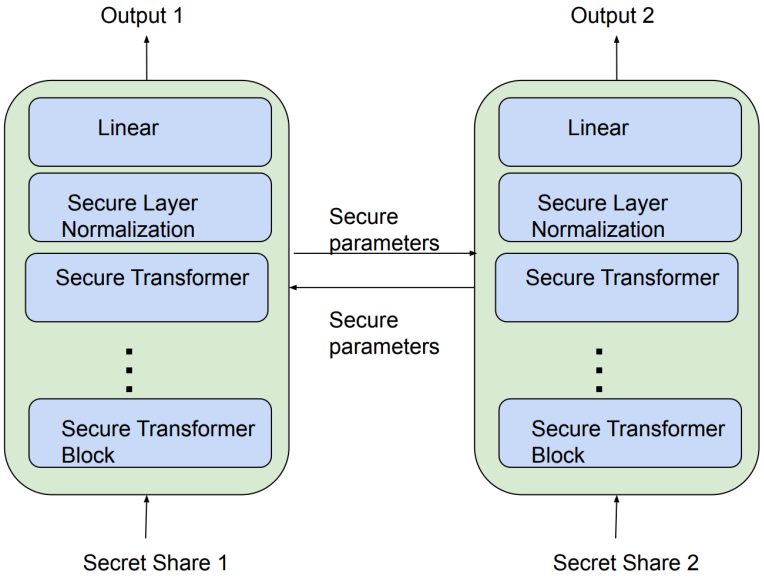

Figure 3: Privacy-Preserving Transformer Model

tion products to ensure the correctness of the layer outputs. These secure parameters are generated by a configuration server and distributed to the relevant servers. At the end of the process, each server produces a partial computation result, denoted as $\boldsymbol{O_i}$, and sends it back to the data owner. The data owner combines the computation outputs and decodes them into the generated text.

3.2. PPGPT Execution Flow

As an illustration of the execution flow of PPGPT, the user provides text input, which an Encoder encodes into a vocabulary of tokens composed of subword units. Both the vocabulary tokens and their positional indexes are then processed by the positional embedding unit and the token embedding unit to map the original input text data to dense vector representations. The input vector $\boldsymbol{t}$ is subsequently split into random secret shares $\boldsymbol{x^{(i)}}$, preserving an additive relationship.

These secure shares are encrypted using a pre-configured encryption algorithm before being transmitted to the servers, ensuring protection during public network transmission. For instance, $\boldsymbol{x^{(1)}}$ is encrypted with AES-512 using an encryption key provided by the Config Server. The encrypted data

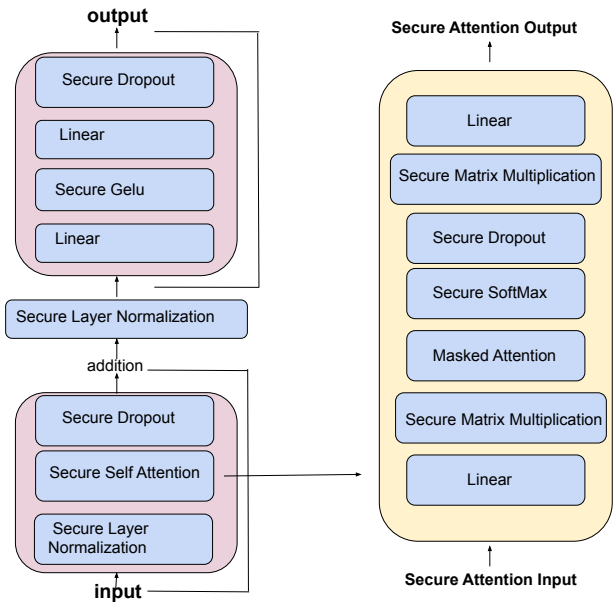

Figure 4: Privacy-Preserving Transformer Layers

is then decrypted on the server side to recover $\boldsymbol{x^{(1)}}$, which is subsequently used as input for PPGPT's privacy-preserving transformer model.

During execution, the servers may exchange secure parameters with each other and receive additional parameters from the Config server as defined by the secure protocol, eventually generating outputs $\boldsymbol{O_i}$. These outputs represent the secret shares of the generated token indices from PPGPT. The outputs are then encrypted again and transmitted back to the user.

Upon receiving the encrypted outputs, the user decrypts them and combines the results by summing the shares. The decoder then maps the combined token indices back to the generated text, resulting in the final output.

4. Design of Privacy-Preserving Transformer Layers

For a layer in a transformer with input tensor data $\boldsymbol{f}$ and output $\boldsymbol{o}$ defined as $\boldsymbol{o} = G(\boldsymbol{f})$, we introduce privacy preserving layers PL_i. Thus, $PL_1(\boldsymbol{f_1}) + PL_2(\boldsymbol{f_2}) + ... + PL_n(\boldsymbol{f_n}) = \boldsymbol{o_1} + \boldsymbol{o_2} + ... + \boldsymbol{o_n} = o$, where i denotes the index of the computation party.

For a linear transformation layer, denoted by G_{linear}, acting on an input tensor $\boldsymbol{f}$, we observe that $G_{linear}(\boldsymbol{f}) = G_{linear}(\boldsymbol{f_1} + \boldsymbol{f_2} + ... + \boldsymbol{f_n}) = G_{linear}(\boldsymbol{f_1}) + G_{linear}(\boldsymbol{f_2}) + ... + G_{linear}(\boldsymbol{f_n})$. This property indicates that the

transformation preserves the additive nature of secret shares. We leverage the original operations within a layer to process a secret share, ensuring consistency with this property. However, such consistency does not extend to non-linear transformations, necessitating specialized design and operations to construct privacy preserving counterparts. Consequently, we have developed privacy preserving versions for all known types of layers utilized in PPGPT. In the subsequent paragraphs, we elucidate our design approach for several key layers.

In this work, we adopt a hybrid approach that combines approximation and precise ciphertext computation to achieve a balance between minimizing accuracy losses and mitigating the prolonged execution time typically associated with secure computations. Specifically, since secure polynomial transformations are efficient [35], we leverage them to facilitate privacy-preserving approximations in activation layers. However, given that transformer architectures include numerous layer normalization layers, we designed a specialized secure function to accurately and efficiently recover the outputs of layer normalization. This design is aligned with other secure functions to maintain consistency and efficiency throughout the network.

This design approach also facilitates the segmentation of the DNN model, as proposed in [36, 37]. As a result, PPGPT allows for splitting certain layers between the client and server sides. This design ensures that only the secret shares of more abstract feature data, rather than raw data, are revealed to the server and public network, thereby enhancing security. Additionally, the level of security can be further improved by increasing the number of computation parties involved.

This strategy empowers us to design a privacy preservation system that exhibits greater resilience against model inversion attacks [38], membership inference attack[39], and other cyber security attacks. By combining horizontal and vertical splitting techniques, we enhance the security and scalability of PPGPT, providing users with a flexible and adaptable solution for preserving the privacy of their data. Table 1 presents the notation used in the subsequent sections.

4.1. Secure GELU

4.1.1. Secure Multiplication

Efficient tensor operations rely heavily on addition and multiplication. In the context of secret-shared inputs that follow an additive relationship, textbfBeaver's Triples [40] are used to enable secure multiplication without

Notation	Description
$\boldsymbol{x}$	Raw data vector
$\boldsymbol{x^{(1)}}, \boldsymbol{x^{(2)}}$	Secret share data vector of raw data
$x_j^{(i)}$	j^{th} element in i^{th} secure share data of $\boldsymbol{x}$
$\boldsymbol{S_i}$	Computation servers
$\boldsymbol{k}, \boldsymbol{h}, \boldsymbol{c}$	Random Beaver's Triple vectors
$\boldsymbol{v_1}, \boldsymbol{v_2}$	Secure Polynomial Transformation outputs
$\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\zeta}, \boldsymbol{\gamma}$	Random vectors for input of secure multiplication computation
μ	Mean of the vector
$\boldsymbol{l}, \boldsymbol{m}, \boldsymbol{\theta}$	$\boldsymbol{l}$ is a random vector, and $\boldsymbol{m}, \boldsymbol{\theta}$ are transformed secret share data vectors in Secure Layer Normalization

Table 1: Notation Table for Secure Protocols

revealing any underlying data. Below, we describe the secure multiplication protocol executed by n servers $\{S_1, S_2, \ldots, S_n\}$ over secret shares of values x and y.

Stage 1: Pre-computation. A configuration server generates a random Beaver Triple (k, h, c), where $c = k \cdot h$. These are split into secret shares $(k^{(i)}, h^{(i)}, c^{(i)})$, satisfying:

$$\sum_{i=1}^{n} k^{(i)} = k, \quad \sum_{i=1}^{n} h^{(i)} = h, \quad \sum_{i=1}^{n} c^{(i)} = c.$$

No individual server knows the full triple.

Stage 2: Masked Difference Computation. Each server S_i receives shares $x^{(i)}$ and $y^{(i)}$ and computes:

$$\Delta x^{(i)} = x^{(i)} - k^{(i)}, \quad \Delta y^{(i)} = y^{(i)} - h^{(i)}.$$

The servers then exchange these masked values and compute:

$$\Delta x = \sum_{i=1}^{n} \Delta x^{(i)} = x - k, \quad \Delta y = \sum_{i=1}^{n} \Delta y^{(i)} = y - h.$$

Stage 3: Secure Multiplication. Each server computes its partial product share:

$$z^{(i)} = c^{(i)} + \Delta x \cdot h^{(i)} + \Delta y \cdot k^{(i)}.$$

The servers then aggregate:

$$x \cdot y = \sum_{i=1}^{n} z^{(i)} + \Delta x \cdot \Delta y.$$

This protocol ensures data privacy by avoiding direct access to raw values, while securely computing the product of two secret shares.

Algorithm 1 Secure Multiplication.

On computation server S_i, $i \in \{1, ..., n\}$
Input: Randomly split vectors $\boldsymbol{x_i}, \boldsymbol{y_i}$ derived from the secret shares $\boldsymbol{x}$ and $\boldsymbol{y}$.
Output: vector $\{\boldsymbol{z_i}\}$

Configuration Server generates a set of random integer vectors $\boldsymbol{k}, \boldsymbol{h}, \boldsymbol{c}$:
$\sum_{i=1}^{n} \boldsymbol{k_i} = \boldsymbol{k}, \quad \sum_{i=1}^{n} \boldsymbol{h_i} = \boldsymbol{h}, \quad \sum_{i=1}^{n} \boldsymbol{c_i} = \boldsymbol{c}$
Masked Value Calculation: S_i computes $\boldsymbol{\Delta x_i} = \boldsymbol{x_i} - \boldsymbol{k_i}, \quad \boldsymbol{\Delta y_i} = \boldsymbol{y_i} - \boldsymbol{h_i}$.
Edge servers aggregate the masked difference:

$$\boldsymbol{\Delta x} = \sum_{i=1}^{n} \boldsymbol{\Delta x_i} = \boldsymbol{x} - \boldsymbol{k}, \quad \boldsymbol{\Delta y} = \sum_{i=1}^{n} \boldsymbol{\Delta y_i} = \boldsymbol{y} - \boldsymbol{h}.$$

S_i computes $\boldsymbol{z_i} = \boldsymbol{c_i} + \boldsymbol{\Delta x} \cdot \boldsymbol{h_i} + \boldsymbol{\Delta y} \cdot \boldsymbol{k_i}$
return $\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{i=1}^{n} \boldsymbol{z_i} + \boldsymbol{\Delta x} \cdot \boldsymbol{\Delta y}$

4.1.2. Secure Polynomial Transformation

In privacy-preserving deep neural networks, non-linear transformations such as layer normalization, GELU, and softmax play critical roles in ensuring model performance. These functions introduce non-linearity, which

is essential for the network's ability to learn complex patterns. However, performing such non-linear operations securely poses challenges when inputs are secret-shared. The Taylor series provides an effective approximation of these non-linear functions by expressing them as polynomial expansions. By rewriting non-linear transformations in polynomial form, the secure computation framework can perform these operations using secure addition and secure multiplication protocols. These protocols are well-established for executing arithmetic on secret-shared inputs without revealing the underlying data.

Our secure polynomial transformation approach accepts two inputs: the first input parameter contains the type of computation (e.g., the target function), and the second one indicates the degree of the Taylor series, which determines the polynomial's complexity and accuracy. The transformation outputs a vector set $[\boldsymbol{v_1}, \boldsymbol{v_2}]$, $\boldsymbol{v_1}$ containing polynomial coefficients and corresponding degrees derived from the raw data approximation, as well as a second vector $\boldsymbol{v_2}$ to compensate for discrepancies introduced by secure computation on secret-shared inputs. $\boldsymbol{v_1}$ and $\boldsymbol{v_2}$ are arrays of polynomial coefficients sorted by degree, which can be used as formulas derived from user data to compute the corresponding calculations, such as the hyperbolic tangent (`tanh`) and sigmoid functions.

These vectors are used to ensure that the final output of the secure computation matches the non-linear transformation applied to the original raw data, leveraging secure addition and multiplication protocols to maintain accuracy and privacy.

For example, consider the non-linear hyperbolic tangent function $\tanh(\boldsymbol{x}) = \frac{e^{\boldsymbol{x}} - e^{-\boldsymbol{x}}}{e^{\boldsymbol{x}} + e^{-\boldsymbol{x}}}$, which is a core component of the Gaussian Error Linear Unit (GELU) activation function commonly used in deep learning. The Taylor series expansion of $\tanh(\boldsymbol{x})$ is given by $\tanh(\boldsymbol{x}) = \sum_{n=0}^{\infty} (-1)^n \frac{2^{2n}(2^{2n}-1)B_{2n}}{(2n)!} \boldsymbol{x}^{2n-1}$, where B_{2n} are the Bernoulli numbers. Using the secure polynomial transformation method with the function type tanh and degree 3, we approximate $\tanh(\boldsymbol{x})$ as $\tanh(\boldsymbol{x}) \approx \boldsymbol{x} - \frac{1}{3}\boldsymbol{x}^3$. This results in a vector $\boldsymbol{v_1}$ containing coefficients $[1, -\frac{1}{3}]$ and corresponding degrees $[1, 3]$.

Now consider two secret shares, $\boldsymbol{x^{(1)}}$ and $\boldsymbol{x^{(2)}}$, such that $\boldsymbol{x} = \boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}}$. The sum of their individual approximations is $\tanh(\boldsymbol{x^{(1)}}) + \tanh(\boldsymbol{x^{(2)}}) = (\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}}) - \frac{1}{3}((\boldsymbol{x^{(1)}})^3 + (\boldsymbol{x^{(2)}})^3)$. The discrepancy between the approximation of $\tanh(\boldsymbol{x})$ and the sum of individual approximations is $\tanh(\boldsymbol{x}) - (\tanh(\boldsymbol{x^{(1)}}) + \tanh(\boldsymbol{x^{(2)}})) = -\frac{1}{3}(\boldsymbol{x}^3 - ((\boldsymbol{x^{(1)}})^3 + (\boldsymbol{x^{(2)}})^3))$. Expanding $\boldsymbol{x}^3 -$

$((\boldsymbol{x^{(1)}})^3+(\boldsymbol{x^{(2)}})^3)$, we get $\boldsymbol{x}^3-((\boldsymbol{x^{(1)}})^3+(\boldsymbol{x^{(2)}})^3) = 3(\boldsymbol{x^{(1)}})^2\boldsymbol{x^{(2)}}+3\boldsymbol{x^{(1)}}(\boldsymbol{x^{(2)}})^2$. Thus, the discrepancy becomes:

$$\tanh(\boldsymbol{x}) - \big(\tanh(\boldsymbol{x^{(1)}}) + \tanh(\boldsymbol{x^{(2)}})\big) = -(\boldsymbol{x^{(1)}})^2\boldsymbol{x^{(2)}} - \boldsymbol{x^{(1)}}(\boldsymbol{x^{(2)}})^2. \quad (2)$$

By using secure multiplication, we can compute the discrepancy value and store it in the returned vector $\boldsymbol{v_2}$.Together, the polynomial approximation vector and the discrepancy vector represent the output of the secure polynomial transformation.

By leveraging the secure multiplication protocol, the discrepancy vector is computed and incorporated into the final result. This ensures that the aggregated output from the secret shares is identical to the non-linear transformation output computed from the raw data.

Algorithm 2 Secure GELU

Computation Server S_i, $i \in \{1, \ldots, n\}$
Input: Secret share of data $\boldsymbol{x^{(i)}}$, polynomial degree $\boldsymbol{n}$
Output: Vector $\{\boldsymbol{k_i}\}$

$\boldsymbol{f(x)}, \boldsymbol{d(x^{(1)}, x^{(2)})} = \text{SecPolyTransform}(\text{GELU}, \boldsymbol{n})$
for each element in $\boldsymbol{d(x^{(1)}, x^{(2)})}$ **do**
    Random split $\boldsymbol{x^{(1)}} = \boldsymbol{\alpha} + \boldsymbol{\beta}$, $\boldsymbol{x^{(2)}} = \boldsymbol{\zeta} + \boldsymbol{\gamma}$
    $\boldsymbol{d} = \boldsymbol{d} + \text{secureMultiply}(\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\zeta}, \boldsymbol{\gamma})$
end for
return $\boldsymbol{f(x^{(i)})} + \boldsymbol{d}$

GELU is a non-linear activation function employed in neural networks, including specific variants of the GPT architecture:

$$\text{GELU}(\boldsymbol{x}) \approx \frac{\boldsymbol{x}}{2}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(\boldsymbol{x} + 0.044715\boldsymbol{x}^3)\right)\right) \quad (3)$$

Suppose we have two secret shares $\boldsymbol{x^{(1)}}$ and $\boldsymbol{x^{(2)}}$ stored on servers S_1 and S_2 respectively. Through the example of tanh in subsection Secure Polynomial Transformation, we know that $\tanh(\boldsymbol{x}) \approx \boldsymbol{x} - \frac{1}{3}\boldsymbol{x}^3$. Let $\boldsymbol{x} = \boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}}, f(\boldsymbol{x}) = \boldsymbol{x} - \frac{1}{3}\boldsymbol{x}^3, g(\boldsymbol{x}) = \boldsymbol{x} + k\boldsymbol{x}^3$ where $k = 0.044715$. Then the difference between $f(g(\boldsymbol{x}))$ and $f(g(\boldsymbol{x^{(1)}})) + f(g(\boldsymbol{x^{(2)}}))$ can be calculated as

$\boldsymbol{d(x^{(1)}, x^{(2)})} = \left(\boldsymbol{k} - \frac{1}{3}\right)\left[(\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}})^3 - (\boldsymbol{x^{(1)}})^3 - (\boldsymbol{x^{(2)}})^3\right] - \boldsymbol{k}\left[(\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}})^5 - (\boldsymbol{x^{(1)}})^5 - (\boldsymbol{x^{(2)}})^5\right] - \boldsymbol{k}^2\left[(\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}})^7 - (\boldsymbol{x^{(1)}})^7 - (\boldsymbol{x^{(2)}})^7\right] - \frac{\boldsymbol{k}^3}{3}\left[(\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}})^9 - (\boldsymbol{x^{(1)}})^9 - (\boldsymbol{x^{(2)}})^9\right]$

$\approx (\boldsymbol{k} - \frac{1}{3})(3\boldsymbol{x^{(1)}}\boldsymbol{x^{(2)}}(\boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}})) = (3\boldsymbol{k} - 1)((\boldsymbol{x^{(1)}})^2\boldsymbol{x^{(2)}} + \boldsymbol{x^{(1)}}(\boldsymbol{x^{(2)}})^2)$.

Given $f(\boldsymbol{x})$ and $\boldsymbol{d(x^{(1)}, x^{(2)})}$, servers S_1 and S_2 compute $f(\boldsymbol{x^{(i)}})$ locally. They then use a secure multiplication protocol to compute $\boldsymbol{d(x^{(1)}, x^{(2)})}$. The resulting discrepancy vector is added to the output of the Secure GELU. The execution flow of the algorithm is presented in Algorithm 2.

4.2. Secure Layer Normalization

Layer normalization [41] is a technique employed in transformer models to stabilize and enhance training by normalizing inputs across features within each layer. Unlike batch normalization, it ensures consistent mean and variance across the sequence, leading to stable activations. This normalization is applied prior to the attention mechanisms and feedforward networks, thereby improving the model's learning efficiency for complex sequences.

The layer normalization can be represented as:

$$\text{LayerNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \tag{4}$$

Since ϵ is constant and γ, β are also constants during the inference process, $\boldsymbol{x}$ represents the raw feature data, and $\boldsymbol{x^{(1)}}, \boldsymbol{x^{(2)}}$ are the secret share data. Additionally, the mean μ can be retrieved by summing the means of the secret share data:

$$\mu = \mu_1 + \mu_2 = \sum_{i=1}^{n} \frac{x_i^{(1)}}{n} + \sum_{i=1}^{n} \frac{x_i^{(2)}}{n} \tag{5}$$

Thus, the core focus is to retrieve the variance of the raw feature data, where:

$$v(\boldsymbol{x}) = \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2 \tag{6}$$

Let S_1 and S_2 compute the variance $v(\boldsymbol{x^{(1)}})$ and $v(\boldsymbol{x^{(2)}})$ locally. The difference for the i-th element can be expressed by:

$$(x_i - \mu)^2 - \left((x_i^{(1)} - \mu)^2 + (x_i^{(2)} - \mu)^2\right) \tag{7}$$

Algorithm 3 Secure Layer Normalization

Computation Server S_i, $i \in 1, ...n$
Input: secret share of Feature vector $\boldsymbol{x^{(1)}}, \boldsymbol{x^{(2)}}$
Output: layer normalization partial result of $\boldsymbol{t_1}, \boldsymbol{t_2}$

S_1, S_2 computes $\bar{\mu} = \bar{\mu_1} + \bar{\mu_2}$
S_1 computes the ratio $\boldsymbol{r}$ for every two neighbor element in $\boldsymbol{x^{(1)}}$
where $r_i = x_i^{(1)} \div x_{i+1}^{(1)}$
S_2 create array $\boldsymbol{l}$
for int $j = 0$; $j < \frac{n}{2}$; $j = j++$ **do**
    $\boldsymbol{l[j]} = rand()$
end for
S_1 forward r to S_2
for for element with in $\boldsymbol{x^{(2)}}$ with index h; $h < n$; $h = h + 2$ **do**
    if h is even **then**
        $\theta[h] = x^{(2)}[h] + L[h]$
    else
        $\theta[h] = x^{(2)}[h] + (-L[h]) * R[h]$
    end if
end for
S_2 forward $\boldsymbol{\theta}$ to S_1
S_1 create empty array $\boldsymbol{m}$
for each element in $x^{(1)}$ **do**
    $\boldsymbol{m} = \boldsymbol{m} + 2 * x_i^{(1)} * \theta_i$
end for
S_1 computes $var_1 = \sum_{i=1}^{n}(x_i^{(1)} - \mu)^2 - n * \mu^2 + \boldsymbol{m}$
S_2 computes $var_2 = \frac{\sum_{i=1}^{n}(x_i^{(2)} - \bar{r})^2}{n}$
S_1, S_2 collaborate compute $var = \sqrt{\frac{(var_1 + var_2)}{n}}$
S_1 return $t_1 = \frac{\boldsymbol{x^{(1)}} - \mu}{Var}$

S_2 return $t_2 = \frac{\boldsymbol{x^{(2)}} - \mu}{Var}$

Substituting $x_i = (x_i^{(1)} + x_i^{(2)})$, we obtain:

$$2x_i^{(1)} \cdot x_i^{(2)} - \mu^2 \tag{8}$$

Thus, the difference can be computed by invoking the secure multiplication protocol. The results are then aggregated to obtain $v(\boldsymbol{x^{(1)}}) + v(\boldsymbol{x^{(2)}})$. As a result, the variance can be computed, and S_1 and S_2 can compute the Secure Layer Normalization outputs.

However, due to the complexity of the secure multiplication protocol and the high frequency of its invocation during the secure batch normalization layer in the inference process, we propose an optimization to accelerate the calculation process. As we know, the variance difference is $\sum_{i=1}^{n} 2x_i^{(1)} \cdot x_i^{(2)} - \mu^2$.

We define a protocol as described in Algorithm 3. First, S_1 computes a ratio vector $\boldsymbol{r}$ for every two neighboring elements in $\boldsymbol{x^{(1)}}$, for example, $r_1 = \frac{x_1^{(1)}}{x_2^{(1)}}$, $r_2 = \frac{x_3^{(1)}}{x_4^{(1)}}$. Here, $x_j^{(i)}$ indicates the j-th element in the i-th secret share of $\boldsymbol{x}$. The ratio vector is forwarded to S_2. S_2 generates a vector of random values $\boldsymbol{l}$ with length equal to $\boldsymbol{r}$. For each pair of elements in $\boldsymbol{x^{(2)}}$, the first one will have the value $x^{(2)} + L$ added, and the second one will have the product of the random vector and the negation of the ratio vector added: $x^{(2)} + (-L) \cdot (-R)$ (3 Line 13-18). The updated vector $\boldsymbol{\theta}$, which contains the transformed $\boldsymbol{x^{(2)}}$ with values from $\boldsymbol{r}$ and $\boldsymbol{l}$, is forwarded to S_1.

By doing this, S_1 can compute the variance difference without using the secure multiplication protocol. For example, $r_1 = \frac{x_1^{(1)}}{x_2^{(1)}}$, and S_2 computes $\theta_1 = x_1^{(2)} + L_1$, $\theta_2 = x_2^{(2)} + (-R_1) \cdot L_1$. Then, S_1 computes $2(x_1^{(1)} \cdot \theta_1 + x_2^{(1)} \cdot \theta_2) = 2(x_1^{(1)} \cdot (x_1^{(2)} + L_1) + x_2^{(1)} \cdot (x_2^{(2)} + (-R_1) \cdot L_1)) = 2x_1^{(1)} \cdot x_1^{(2)} + 2x_1^{(1)} \cdot L_1 + 2x_2^{(1)} \cdot x_2^{(2)} - 2\left(x_2^{(1)} \cdot \frac{x_1^{(1)}}{x_2^{(1)}} \cdot L_1\right) = 2(x_1^{(1)} \cdot x_1^{(2)} + x_2^{(1)} \cdot x_2^{(2)})$, where the random values are eliminated by combining the results. In this way, we can compute the variance difference by adding the additional μ^2.

where the random values are eliminated by combining the results. In this way, we can compute the variance difference by adding the additional μ^2.

It is worth noting that the optimized Secure Layer Normalization works for non-zero vector arrays, and padding is applied for vectors with an odd size. This optimization reduces the number of secure multiplication invocations, and our experimental results demonstrate a significant performance improvement.

Algorithm 4 Secure Softmax

Computation Server S_i, $i \in \{1, ...n\}$
Input: secret share of data $\boldsymbol{x^{(i)}}$, polynomial degree $\boldsymbol{n}$
Output: vector $\boldsymbol{\{k_i\}}$

$\boldsymbol{f(x), d(x^{(1)}, x^{(2)})} = \text{SecPolyTransform}(\text{softMax}, \boldsymbol{n})$
for for element within $\boldsymbol{d(x^{(1)}, x^{(2)})}$ **do**
    random split $\boldsymbol{x^{(1)} = \alpha + \beta, x^{(2)} = \zeta + \gamma}$
    $\boldsymbol{d = d} + \text{secureMultiply}\boldsymbol{(\alpha, \beta, \zeta, \gamma)}$
end for
$\boldsymbol{u^{(i)} = f(x^i) + d(x^{(1)}, x^{(2)})}$
$\boldsymbol{S_1, S_2}$ exchange $\boldsymbol{u^{(1)}, u^{(2)}}$ and calculate $\boldsymbol{u = u^{(1)} + u^{(2)}}$
for each element with index $\boldsymbol{h}$ in $\boldsymbol{x^{(i)}}$ **do**
    Return $\frac{u_h^i}{u}$
end for

Softmax is a mathematical function used to convert raw scores or logits into probabilities with the representation of $\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{i=1}^{N} e^{x_i}}$. This conversion facilitates tasks like language modeling and text generation, where predicting the next word requires a probabilistic approach. Moreover, softmax aids in stabilizing training by ensuring non-negative probabilities that sum to 1, crucial for calculating cross-entropy loss during backpropagation.

Similar to Secure GELU, the Secure Polynomial Transformation in Algorithm 4 returns the polynomial function of $\boldsymbol{f(x)} = e^{\boldsymbol{x}} = \sum_{\boldsymbol{n}=0}^{\infty} \frac{\boldsymbol{x}_n}{\boldsymbol{n}!}$ along with the discrepancy vector. For instance, when $\boldsymbol{n} = 3$, $e^{\boldsymbol{x}} \approx 1 + \boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}} + \frac{(\boldsymbol{x^{(1)}})^2}{2} + \frac{(\boldsymbol{x^{(2)}})^2}{2} + \boldsymbol{x^{(1)}x^{(2)}}$, and the discrepancy is computed as

$$\begin{aligned}
\boldsymbol{d(x^{(1)}, x^{(2)})} &= e^{\boldsymbol{x}} - \left(e^{\boldsymbol{x^{(1)}}} + e^{\boldsymbol{x^{(2)}}}\right) \\
&= \left(1 + \boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}} + \frac{(\boldsymbol{x^{(1)}})^2}{2} + \frac{(\boldsymbol{x^{(2)}})^2}{2} + \boldsymbol{x^{(1)}x^{(2)}}\right) \\
&\quad - \left(2 + \boldsymbol{x^{(1)}} + \boldsymbol{x^{(2)}} + \frac{(\boldsymbol{x^{(1)}})^2}{2} + \frac{(\boldsymbol{x^{(2)}})^2}{2}\right) \\
&= \boldsymbol{x^{(1)}x^{(2)}} - 1
\end{aligned} \tag{9}$$

The discrepancy vector $\boldsymbol{d(x^{(1)}, x^{(2)})}$ can then be retrieved using a secure multiplication protocol.

For each secret share data with index $\boldsymbol{h}$ in $\boldsymbol{x^{(1)}}, \boldsymbol{x^{(2)}}$, $\boldsymbol{S_1}$ computes $\boldsymbol{f(x_h^{(1)})}+\boldsymbol{d(x_h^{(1)}, x_h^{(2)})}$ and stores the result in the vector $\boldsymbol{u_h^{(1)}}$. Similarly, $\boldsymbol{S_2}$ computes $\boldsymbol{f(x_h^{(2)})}$ and stores the result in the vector $\boldsymbol{u_h^{(2)}}$. Both $\boldsymbol{S_1}$ and $\boldsymbol{S_2}$ then collaborate to compute the summation $\boldsymbol{u} = \boldsymbol{u^{(1)}} + \boldsymbol{u^{(2)}} = \sum_{\boldsymbol{h}=1}^{\boldsymbol{N}} e^{x_h}$. Finally, the softmax result for $\boldsymbol{S_1}$ is $\frac{\boldsymbol{u_h^{(1)}}}{\boldsymbol{u}}$, and for $\boldsymbol{S_2}$, it is $\frac{\boldsymbol{u_h^{(2)}}}{\boldsymbol{u}}$.

4.4. Secure Dropout

In the transformer architecture, dot product attention calculates attention weights between tokens in the input sequence. Dot dropout is a variant of dropout applied during dot product attention computation. Dropout is a regularization technique employed in neural networks to mitigate overfitting. In secure dropout, as the input maintains an additive relationship, after S_1 randomly replaces specific input data with zero values, it forwards these indices to S_2, ensuring dropped-out values correspond to the same locations in both secure shares. Consequently, the output of secure dropout preserves the additive relationship between the raw data outputs.

5. Performance Evaluation

To assess the performance of PPGPT, we conduct experiments on a client-server testbed. The user system runs on MacOS Catalina, featuring an 8-Core Intel Core i9 processor clocked at 2.3 GHz, an NVIDIA GeForce GTX 1660 Super GPU, and 16 GB of 2667 MHz DDR4 memory. Each PPGPT server in the testbed is powered by an AMD Ryzen 7 processor with 6 cores running at 3.2 GHz, equipped with 16 GB of DRAM, and operating on Ubuntu Linux v20.04 with Python v3.8.

We implement PPGPT using a lightweight GPT-2 model (miniGPT) with privacy-preserving inference through secret sharing ($n = 2$). Byte Pair Encoding (BPE) tokenizes input text, and tokens are processed by the PPGPT positional and token embedding units. The input vector is split into secret shares $x^{(1)}$ and $x^{(2)}$, encrypted with AES-512, and sent to separate servers (E_1, E_2). Each server performs privacy-preserving inference, and partial logits are combined by a config server using greedy sampling. The final output is retrieved by the data owner by adding the outputs from S_1 and S_2.

Through these experiments, our objective is to evaluate the performance and accuracy between PPGPT and the base model. The experiment environment enables us to assess latency, system resource utilization, and the privacy

preservation of input data. The PPGPT model parameters are configured with a pre-trained miniGPT with six transformer blocks and trained on the WikiText-103 dataset, a widely used resource in LLM research. We utilize PyTorch v0.4.0 and Torch v0.20 to implement privacy-preserving Transformer layers in PPGPT. We conduct training with one epoch, vocabulary size 50257, batch size of 32, and a learning rate of $5e^{-4}$. The pre-trained miniGPT serves as the baseline model for experimental comparison, with both PPGPT and miniGPT sharing identical layer parameters, vocabulary, encoder, decoder, and sampling strategy (greedy).

Configuration	**General**	**News**	**Technical**	**Legal**
average input length 20	3.43×10^{-5}	5.74×10^{-5}	1.2×10^{-6}	4.44×10^{-5}
average input length 40	1.2×10^{-5}	9.88×10^{-4}	3.3×10^{-6}	1.63×10^{-5}
average input length 60	2.48×10^{-5}	0.32×10^{-6}	3.31×10^{-6}	1.26×10^{-4}
average input length 80	5.1×10^{-5}	7.22×10^{-4}	0.82×10^{-5}	3.78×10^{-5}

Table 2: PPGPT vs Base Model Inference Accuracy

5.1. PPGPT Model Accuracy

The paper presents text generation results using the WikiText-103 test dataset, focusing on four major topics to demonstrate the accuracy of the PPGPT model. The experiment shows that PPGPT achieves an identical level of accuracy, with an average logits score difference in inference output of approximately e^{-5} between the two models when comparing the same state of the two models with identical input tokens (Table 2).

Beyond raw logit similarity, we compare PPGPT's continuations to those generated by the base miniGPT using identical prompts. PPGPT achieves an average BLEU-2 score of 0.93, while the base model scores 0.95, indicating that secure computation introduces only minimal semantic deviation. When evaluating against the ground-truth continuations from the WikiText-103 dataset, both models yield BLEU-2 scores.

Additionally, we compare the logits scores of Secure Layer Normalization, which utilizes a secure polynomial transformation, with our current method. The results indicate that not using secure polynomial transformation leads to an 11.4% increase in accuracy. We verify that the text generated by the secure model achieves a similar level of coherence and relevance compared to the base model, ensuring consistent quality in text generation.

Compared to HE-BERT [48], which applies fully homomorphic encryption to BERT inference and reports a 2–4% drop in classification accuracy with latency exceeding 200 seconds for 10-token inputs, PPGPT maintains high semantic fidelity with a BLEU-2 score of 0.93 and a negligible logits difference of e^{-5}, while completing inference in 1.98 seconds. In contrast, Just Fine-Tune Twice (JFT) [46] focuses on training-time privacy and observes a perplexity increase from 18.1 to 18.6 on WikiText-2, whereas PPGPT preserves inference-time privacy without introducing noticeable degradation in perplexity or output quality.

These results validate the effectiveness of the PPGPT design and implementation, demonstrating its ability to perform text generation tasks with comparable accuracy to the base model. Additionally, PPGPT utilizes the same architecture as the base model, enabling it to be fine-tuned with additional datasets for various NLP tasks like text summarization and question answering.

	MiniGPT		**PPGPT**
Linear	3.7 ms	Linear	8.28 ms
Layer Normalization	8.7 ms	Layer Normalization	0.756 s
GELU	6.2 ms	Secure GELU	0.32 s
Dropout	1.2 ms	Secure Dropout	1.9 ms
Softmax	0.4 ms	Secure Softmax	0.3 s
Masked Attention	1 ms	Secure Masked Attention	1.78 ms
Attention Multiplication	2.6 ms	Secure Attention Multiplication	0.37 s
Total	**23.8 ms**	**Total**	**1.978 s**

Table 3: PPGPT vs Base Model Execution Time Breakdown

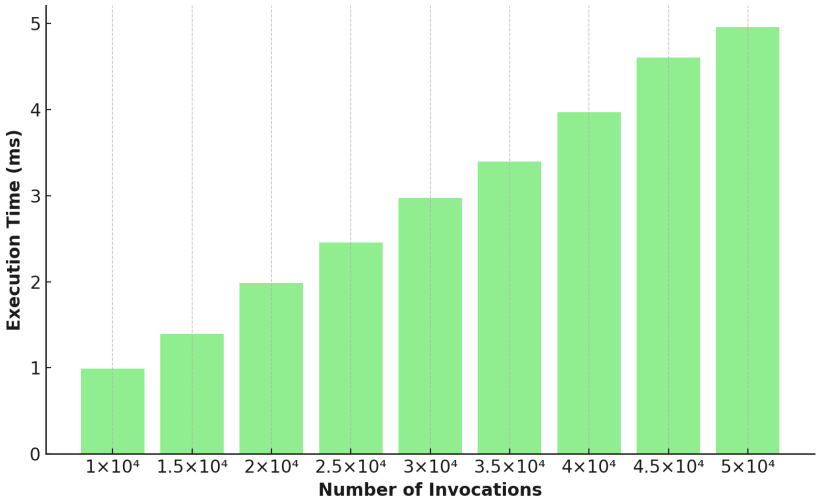

Figure 5: Execution Time of Secure Multiplication

5.2. Privacy Enhancement for User Data

Additive secret sharing provides strong resistance against attacks such as man-in-the-middle attacks, side-channel attacks [42, 43], and server memory scraping [44], by transforming raw data into multiple secret shares such that no single share—or subset of shares—reveals the original data without complete reconstruction. During execution, secret shares are exchanged as ciphertext, and servers interact only with randomized intermediate computational parameters under secure protocols, rendering raw data unrecoverable. Moreover, while input token vectors in machine learning models can often be reverse-engineered to reconstruct the original raw text, PPGPT processes only the secret shares of token vectors, never exposing actual token values in memory or communication. This further strengthens the privacy guarantees, ensuring that neither raw input nor intermediate representations are accessible at any stage. Through the adoption of additive secret sharing, PPGPT inherits these enhanced security properties and offers significantly stronger protection compared to the base model, where raw data and token vectors remain directly exposed in memory or transmission channels.

5.3. PPGPT Inference Performance

PPGPT uses an average of 1.9 seconds to complete an inference process, while the base model averages 23.8 milliseconds (Table 3). The extended inference time is attributed to the execution of privacy-preserving Transformer layers in multiple privacy preserving transformer blocks and the secret share

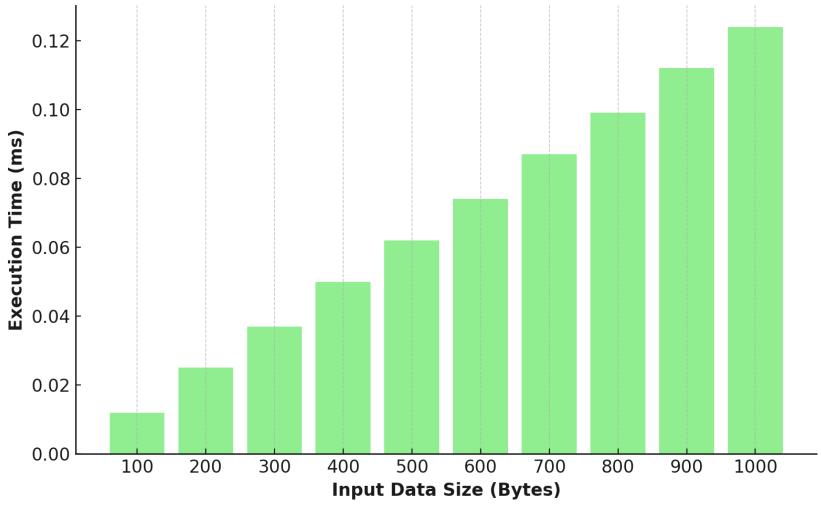

Figure 6: Execution Time of Secure GELU

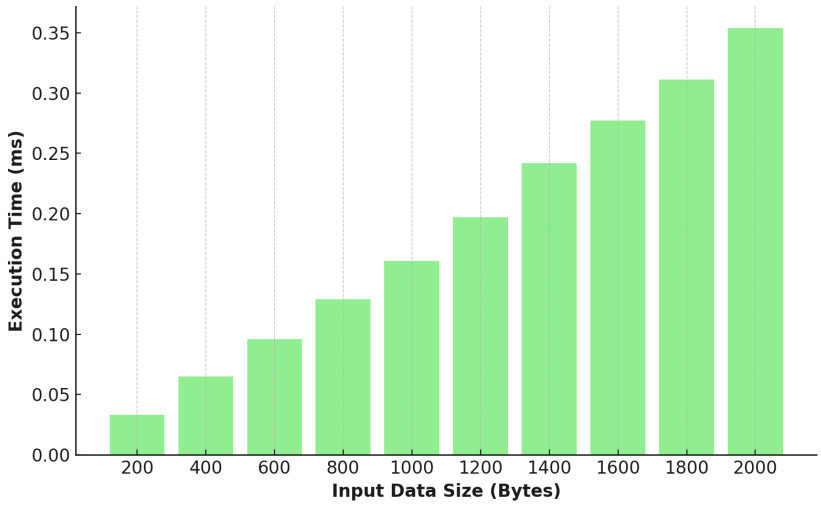

Figure 7: Execution Time of Secure Softmax

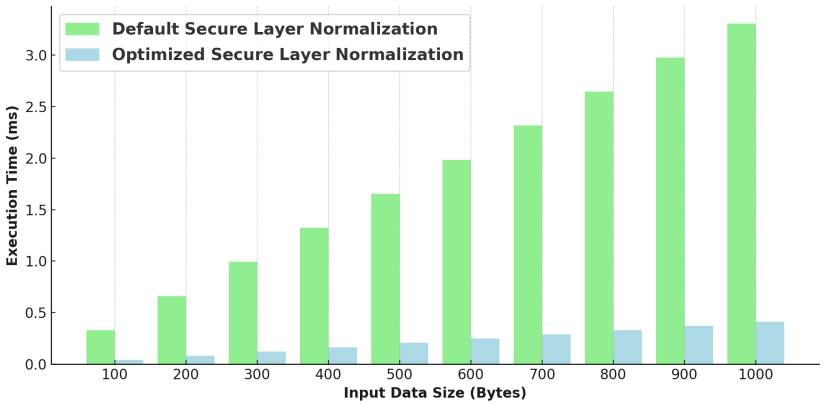

Figure 8: Execution Time of Secure Layer Normalization with Secure Multiplication vs. Optimized Secure Layer Normalization

data generation. In Figure 5, we present the performance of secure multiplication protocols, highlighting the relationship between execution time and the number of invocations. Additionally, Figures 6 and 7 depict the execution time of secure GELU and secure softmax with varying input data sizes in bytes. Notably, secure softmax exhibits comparatively longer execution times, primarily due to its additional iterations on the dataset.Moreover, both privacy-preserving GELU and softmax functions experience extended execution times when handling larger input data sizes due to the increased number of secure multiplication operations required.

As shown in Figure 8, the optimized Secure Layer Normalization is, on average, eight times faster than the default method. This substantial speedup is primarily due to the reduction in the number of secure multiplication invocations. In the default method, secure multiplications are frequently used to compute the variance, which increases computational overhead. In contrast, the optimized version leverages a ratio vector and a random value vector to efficiently compute the variance difference without the need for these multiplications. By eliminating the repetitive secure multiplications, the optimized approach reduces computational complexity, resulting in faster processing times while maintaining privacy and accuracy. Additionally, padding for odd-sized vectors further enhances the consistency and efficiency of the computation.

Figure 9 illustrates a total of 74 bytes of extra data generated during each inference process. Of this, 18 bytes are produced from the computation process during the execution of secure Transformer layers, while 56 bytes

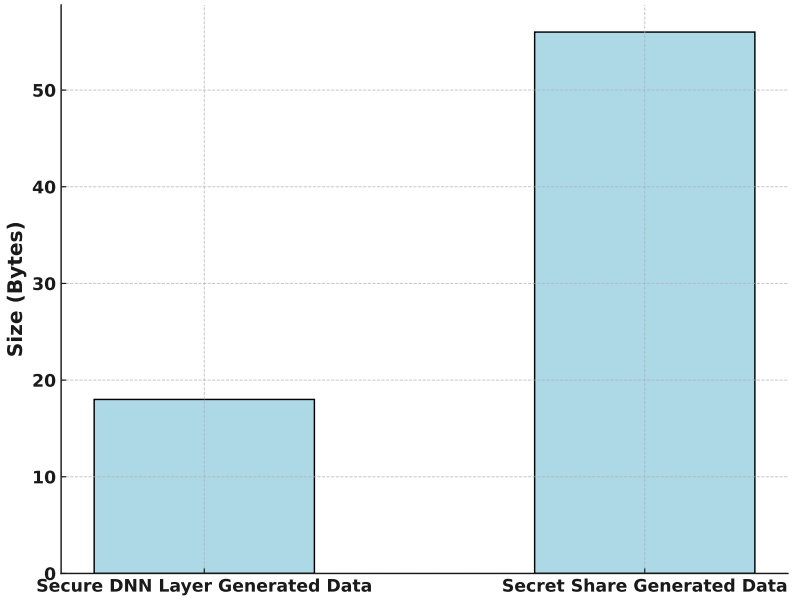

Figure 9: Extra Data (in Bytes) Generated by PPGPT

correspond to the secret share data generated for the tokenized text data.

We analyze the communication overhead introduced by the secret sharing framework in PPGPT, where secure computations require multiple rounds of interaction between servers. Our measurements show that communication-related delay, including secure share exchange and parameter synchronization, accounts for approximately 41% of the total inference latency. The reported total latency of 1.98 seconds includes both computation and communication across two servers. Although this overhead is nonnegligible, it is reduced by several design choices. First, we utilize a lightweight GPT architecture (miniGPT), which decreases the volume of data exchanged. Second, tensor data is processed in batches, allowing communication costs to be distributed across multiple tokens. Third, we introduce optimized secure protocols such as Secure Layer Normalization, which substantially reduces the number of communication rounds. These factors together help keep communication costs practical during privacy-preserving inference with PPGPT.

5.4. Scalability Discussion

PPGPT is designed to be modular and scalable, enabling support for longer input sequences and a greater number of computation servers. As the input token length grows, the number of secure multiplications, par-

ticularly in non-linear layers such as GELU and layer normalization—also increases, resulting in higher latency. Additionally, scaling to more servers can enhance security by distributing secret shares more broadly, but it also introduces greater coordination complexity and communication overhead. In future work, we aim to mitigate these challenges by incorporating parallelized secret sharing protocols and exploring pipeline execution strategies to further improve inference efficiency and scalability.

6. Related Work

In recent advancements, the concept of Selective Differential Privacy (SDP) [45] has been introduced to enhance privacy-preserving mechanisms in LLMs. For example, the framework *Just Fine-tune Twice* (JFT) [46] realized this approach. JFT strategically fine-tunes LLMs in two phases: initially on the entire dataset and subsequently on sensitive data to ensure robust privacy guarantees without compromising model performance. This method stands out by balancing the trade-off between privacy and utility, demonstrating that selective application of differential privacy can effectively protect sensitive information in LLMs. Similarly, Zachary Charles' work [47] also applies DP-SGD (Differentially Private Stochastic Gradient Descent) and explores scalable methods for training LLMs while ensuring user-level privacy. Additionally, data masking technologies are popular in the privacy-preserving field for LLMs. For example, InforMask [12] introduces an innovative unsupervised masking strategy for language model pretraining. By intelligently selecting informative tokens to mask, it significantly outperforms random masking on knowledge-intensive tasks. The resulting InformBERT model achieves competitive performance with larger models, demonstrating the potential of smart masking strategies to enhance language models' knowledge acquisition during pretraining. While these studies are effective at safeguarding training data, PPGPT focuses on protecting inference data to ensure user privacy.

Fully Homomorphic Encryption (FHE) is a mainstream technology for protecting data privacy during model inference for Convolutional Neural Networks (CNNs), but there are few applications of the technology on transformers due to the architectural differences between CNNs and transformers. Notably, there is a work integrating FHE with Large Language Models (LLMs) such as BERT [48]. In the study titled *HE-BERT: Homomorphically Encrypted BERT*, the authors examine the impact of applying FHE on the

inference performance of the BERT model. The findings demonstrate a linear correlation between the number of tokens processed and the execution time, indicating a substantial computational overhead. Specifically, processing 10 tokens incurs a latency of approximately 200 seconds, while 40 tokens require around 600 seconds. Compared to PPGPT, our work achieves an average 172 times speed up, making PPGPT more feasible for real-world applications.

Related efforts have explored privacy-preserving techniques for edge computing and perception networks, including secure data sharing for autonomous vehicles [49], privacy-aware object detection using secure CNNs [50], and end-to-end frameworks for privacy-preserving vehicle-edge perception [51, 52, 53].

7. Conclusion

This paper introduces PPGPT, a privacy-preserving framework designed to safeguard user data while utilizing modern transformer-based LLM. Leveraging a blend of technologies including additive secret sharing, Beaver's tuple, and Taylor series, the framework aims to protect user data during computation and transmission. The architecture, secure functions, and privacy-preserving Transformer layers are thoroughly detailed, and experimental results are presented to demonstrate the effectiveness of the framework.

Despite the effectiveness of PPGPT, several challenges remain in the current design. Specifically, the system is vulnerable to risks if servers collude, as collaborative investigation between servers could potentially expose user data. Additionally, the current design introduces overhead in terms of requiring extra servers, as well as increased memory and disk space usage. As part of future work, we plan to enhance the system's resilience against server collaboration risks by incorporating advanced cryptographic techniques and secure computation methods. Furthermore, we aim to improve system efficiency by optimizing memory consumption and reducing inference latency, making PPGPT more practical and scalable for real-world deployments.

8. Acknowledgment

This work is supported in part by the U.S. National Science Foundation (NSF) under grants CNS-2231519, CNS-2113805, OAC-2017564, CNS-2037982, DUE-2225229 and CCF-2447834.

References

[1] Leslie Lamport, *LaTeX: a document preparation system*, Addison Wesley, Massachusetts, 2nd edition, 1994.

[2] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever, *Improving Language Understanding by Generative Pre-Training*, OpenAI preprint, 2018.

[3] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing, *CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy*, International Conference on Machine Learning (ICML), pp. 201–210, 2016.

[4] Tim van Elsloo, Giorgio Patrini, and Hamish Ivey-Law, *SEALion: A Framework for Neural Network Inference on Encrypted Data*, arXiv preprint arXiv:1904.12840, 2019.

[5] Rachel Cummings, Damien Desfontaines, David Evans, Roxana Geambasu, Yangsibo Huang, Matthew Jagielski, Peter Kairouz, Gautam Kamath, Sewoong Oh, Olga Ohrimenko, Nicolas Papernot, Ryan Rogers, Milan Shen, Shuang Song, Weijie Su, Andreas Terzis, Abhradeep Thakurta, Sergei Vassilvitskii, Yu-Xiang Wang, Li Xiong, Sergey Yekhanin, Da Yu, Huanyu Zhang, and Wanrong Zhang, *Advancing Differential Privacy: Where We Are Now and Future Directions for Real-World Deployment*, Harvard Data Science Review, January 2024.

[6] Jessica Smith, David Williams, and Emily Brown, *Scalable Differential Privacy Mechanisms for Real-Time Machine Learning Applications*, arXiv preprint arXiv:2410.02462, September 2024.

[7] Maryam Aliakbarpour, Syomantak Chaudhuri, Thomas A. Courtade, Alireza Fallah, and Michael I. Jordan, *Enhancing Feature-Specific Data Protection via Bayesian Coordinate Differential Privacy*, arXiv preprint arXiv:2410.18404, October 2024.

[8] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang, *Deep Learning with Differential Privacy*, Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 308–318, 2016.

[9] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith, *Calibrating Noise to Sensitivity in Private Data Analysis*, Proceedings of the 3rd Theory of Cryptography Conference (TCC), pp. 265–284, 2006.

[10] Jinyuan Dong, Jinbo Bi, and Tingting Chen, *Differential Privacy in Machine Learning: A Survey*, IEEE Transactions on Knowledge and Data Engineering (TKDE), 2021.

[11] Josep Domingo-Ferrer and Vicenç Torra, *A Quantitative Comparison of Disclosure Control Methods for Microdata*, in *Confidentiality, Disclosure, and Data Access: Theory and Practical Applications for Statistical Agencies*, Elsevier, pp. 111–133, 2002.

[12] Nafis Sadeq, Canwen Xu, and Julian McAuley, *InforMask: Unsupervised Informative Masking for Language Model Pretraining*, arXiv preprint arXiv:2210.11771, 2022.

[13] Joon Kim and Sejin Park, *Random Gradient Masking as a Defensive Measure to Deep Leakage in Federated Learning*, arXiv preprint arXiv:2408.08430, August 2024.

[14] David Gerault, Anna Hambitzer, Eyal Ronen, and Adi Shamir, *How to Securely Implement Cryptography in Deep Neural Networks*, IACR Cryptology ePrint Archive, Report 2025/288, February 2025.

[15] Arslan Shafique, Abid Mehmood, Moatsum Alawida, Mourad Elhadef, and Mujeeb Ur Rehman, *A Fusion of Machine Learning and Cryptography for Fast Data Encryption Through the Encoding of High and Moderate Plaintext Information Blocks*, Multimedia Tools and Applications, vol. 84, pp. 5349–5375, March 2025.

[16] Wazib Ansar, Saptarsi Goswami, and Amlan Chakrabarti, *A Survey on Transformers in NLP with Focus on Efficiency*, arXiv preprint arXiv:2406.16893, June 2024.

[17] Zhenyu Zhang, Yujia Qin, Zhenzhong Lan, and Jiwei Li, *Attention Heads of Large Language Models: A Survey*, arXiv preprint arXiv:2409.03752, September 2024.

[18] Tri Dao and Albert Gu, *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality*, arXiv preprint arXiv:2405.21060, May 2024.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, *Attention is All You Need*, Advances in Neural Information Processing Systems, pp. 5998–6008, 2017.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1), pp. 4171–4186, 2019.

[21] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al., *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.

[22] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu, *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[23] Charu C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Springer, 2018.

[24] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd Edition), O'Reilly Media, 2022.

[25] Jürgen Schmidhuber, *Deep Learning in Neural Networks: An Overview*, Neural Networks, vol. 61, pp. 85–117, 2015.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, *Attention Is All You Need*, Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.

[27] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler, *Efficient Transformers: A Survey*, ACM Computing Surveys (CSUR), vol. 55, no. 6, pp. 1–28, 2022.

[28] Dilip Kumar Rao and Francis Bach, *Attention Mechanisms: Fundamentals, Research Trends, and Challenges*, Foundations and Trends in Machine Learning, vol. 15, no. 5, pp. 337–500, 2022.

[29] Oded Goldreich, *Foundations of Cryptography: Volume 2, Basic Applications*, Cambridge University Press, 2009.

[30] Moni Naor and Adi Shamir, *Visual Cryptography*, Springer, pp. 1–12, 1994.

[31] Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft, *Unconditionally Secure Constant-Rounds Multi-Party Computation for Equality, Comparison, Bits and Exponentiation*, Theory of Cryptography Conference, Springer, pp. 285–304, 2006.

[32] Adi Shamir, *How to Share a Secret*, Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[33] Yehuda Lindell, *Secure Multiparty Computation*, Communications of the ACM, vol. 64, no. 1, pp. 86–96, 2020.

[34] Ronald Cramer, Ivan Damgård, and Jesper Buus Nielsen, *Secure Multiparty Computation and Secret Sharing*, Cambridge University Press, 2015.

[35] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan, *GAZELLE: A Low Latency Framework for Secure Neural Network Inference*, 27th USENIX Security Symposium (USENIX Security 18), pp. 1651–1669, 2018.

[36] Nicolas Papernot, Martín Abadi, Úlfar Erlingsson, Ian Goodfellow, and Kunal Talwar, *Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data*, International Conference on Learning Representations (ICLR), 2017.

[37] Amirhosein Emami Eshratifar, Mohammad Sadegh Abrishami, and Massoud Pedram, *Bottlenet: A Deep Learning Architecture for Intelligent Mobile Cloud Computing Services*, Proceedings of the 2019 on Great Lakes Symposium on VLSI, pp. 235–240, 2019.

[38] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart, *Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures*, Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, pp. 1322–1333, 2015.

[39] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov, *Membership Inference Attacks Against Machine Learning Models*, 2017 IEEE Symposium on Security and Privacy (SP), pp. 3–18, 2017.

[40] Donald Beaver, *Efficient Multiparty Protocols for Secure Computations*, Proceedings of the 3rd ACM Conference on Computer and Communications Security (CCS '91), pp. 11–18, 1991.

[41] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton, *Layer Normalization*, arXiv preprint arXiv:1607.06450, 2016.

[42] Paul Kocher, Joshua Jaffe, and Benjamin Jun, *Differential Power Analysis*, Advances in Cryptology—CRYPTO'99, Springer, pp. 388–397, 1999.

[43] Mengjia Yan, Yinchi Zhang, Xueqi Zhang, Qijing Dong, Dingyi Ji, Song Lu, Haoti Lyu, Yulong Zhang, and Ruby B. Lee, *Cache Telepathy: Accelerating Cryptanalysis Using Neural Network Assisted Cache Attacks*, 29th USENIX Security Symposium (USENIX Security 20), pp. 1821–1838, 2020.

[44] Ron Lyda and James Hamrock, *Using Entropy Analysis to Find Encrypted and Packed Malware*, IEEE Security and Privacy, pp. 40–45, 2007.

[45] Anupam Ghosh, Alex Golovnev, Adam Smith, and Jinhui Xu, *Selective Differential Privacy*, Proceedings of the 38th International Conference on Machine Learning (ICML), pp. 3749–3759, 2021.

[46] Weiyan Shi, Ryan Shea, Si Chen, Chiyuan Zhang, Ruoxi Jia, and Zhou Yu, *Just Fine-Tune Twice: Selective Differential Privacy for Large Language Models*, arXiv preprint arXiv:2204.07667, 2022.

[47] Zachary Charles, Arun Ganesh, Ryan McKenna, H. Brendan McMahan, Nicole Mitchell, Krishna Pillutla, and Keith Rush, *Fine-Tuning Large Language Models with User-Level Differential Privacy*, arXiv preprint arXiv:2407.07737, 2024.

[48] Lorenzo Rovida and Alberto Leporati, *Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-tiny*, Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics, pp. 3–13, 2024.

[49] Tianyu Bai, Qing Yang, and Song Fu, *User-Defined Privacy Preserving Data Sharing for Connected Autonomous Vehicles Utilizing Edge Computing*, Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing, pp. 145–157, 2023.

[50] Tianyu Bai, Song Fu, and Qing Yang, *Privacy-preserving Object Detection with Secure Convolutional Neural Networks for Vehicular Edge Computing*, Future Internet, vol. 14, no. 11, article 316, 2022.

[51] Tianyu Bai, Danyang Shao, Ying He, Song Fu, and Qing Yang, *P^3: A Privacy-Preserving Perception Framework for Building Vehicle-Edge Perception Networks Protecting Data Privacy*, Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN), pp. 1–10, IEEE, 2023.

[52] Tianyu Bai, Haili Wang, Jingda Guo, Xu Ma, Mahendra Talasila, Sihai Tang, Song Fu, and Qing Yang, *Online Self-Evolving Anomaly Detection for Reliable Cloud Computing*, Proceedings of the 15th IEEE/ACM International Conference on Utility and Cloud Computing (UCC), December 2022.

[53] Xu Ma, Jingda Guo, Andrew Sansom, Mara McGuire, Andrew Kalaani, Qi Chen, Sihai Tang, Qing Yang, and Song Fu, *Spatial Pyramid Attention for Deep Convolutional Neural Networks*, IEEE Transactions on Multimedia, 2021.